Assignment 4 – City Search & Spell-Check: DP and String Algorithms

Author: name surname, group XX
Course: Design and Analysis of Algorithms

1. Experiment Design Card

> Written **before the first run** (see git history: the commit `docs(report): add experiment design card` comes before `feat(experiment)`). Changes made later are listed in the section "Changes to the card".

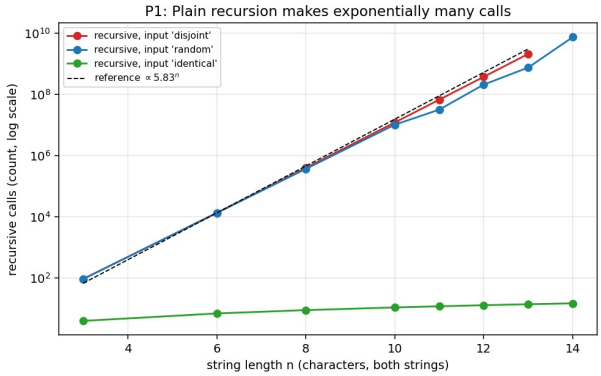
Card field	Content
Question – Edit Distance	(1) Does the plain recursion really explode exponentially, and at which n does it become unusable? (2) Do table and two-row versions grow like `n·m` and does the input change their cost? (3) What does the two-row version save in memory and does it cost time?
Question – String search	(1) Does the naive search really become quadratic on its worst input while KMP stays linear? (2) Is KMP ever worse than naive on "normal" (random) text?
Sizes (n) – Edit Distance	Recursive: n = 3, 6, 8, 10, 11, 12, 13, 14 (stop when one run is longer than ~8 s). The largest is more than 4× the smallest; exponential growth means that larger n is not possible, 14 chars is already about 10^9–10^{10} calls for bad input. Table: n = 250, 500, 1000, 2000, 4000 (a 4000×4000 int table is 64 MB, still fine). Two rows: the same sizes plus 8000 and 16000, where the table would need 256 MB / 1 GB. Both strings have length n.
Sizes (n) – Search	Text length n = 10 000, 20 000, 40 000, 80 000, 160 000 (16×). Pattern length m = n/10, so that the naive worst case $(n-m+1)\cdot m \approx 0.09\cdot n^2$ is clearly visible, but the biggest run is still only ~$2\cdot10^9$ comparisons (a few seconds).
Inputs – Edit Distance	`random`: both strings random over 26 letters (typical case, e.g. two typed street names). `identical`: a = b (best case for the recursion, which then makes only n+1 calls; shows that the DP does not "notice" it). `disjoint`: a uses only letters a–m, b only n–z, so no character ever matches – the worst case for the recursion (all three branches are taken in every call) and the case with the most "replace" decisions for the DP.
Inputs – Search	`random`: random text and random pattern over alphabet {A,C,G,T} (typical log file; the pattern almost never matches for long). `best_naive`: text = A…A, pattern starts with a letter that is not in the text (naive does exactly 1 comparison per start). `worst_naive`: text = A…A, pattern = A…AB (naive compares m chars at every start and fails at the last one; KMP's lps = 0,1,2,… so it has to fall back many times – tests the "≤ 2n" claim). Generator: own Gen class, `java.util.Random` with fixed seed `2026 + 31·n + type-id`.
Metrics	*Edit Distance:* `work` = number of recursive calls (recursive) or number of DP cells computed (table / two rows) – this is the cost the theory talks about; `mem_cells` = number of int cells allocated (memory); `time_ms` (5 runs per point, 3 for the recursion; median is used in the plots). *Search:* `comparisons` = text-char vs pattern-char comparisons in the search phase (the cost in the lecture); `lps_comparisons` = comparisons spent building the LPS array (to see that preprocessing is small); `matches`; `time_ms` (5 runs, median). All counters are increased inside the algorithms.
Prediction	**Recursion:** calls on `disjoint`/`random` grow by a factor ≈ 5.8 for every +1 of n (Delannoy numbers), so a straight line on a log-y plot; on `identical` calls = n+1 (linear). Practical limit n ≈ 12–13 for 8 s. **Table / two-row:** `work` = n^2 exactly, for every input type; time ×4 when n ×2 (slope 2 in log-log), identical for all input types up to small noise; two rows a bit faster or equal (less memory traffic), and `mem_cells` n^2 vs 2n. **Naive:** `random` ≈ 1.3·n comparisons (alphabet 4), `best_naive` ≈ n−m+1, `worst_naive` ≈ $0.09\cdot n^2$ (slope 2). **KMP:** between n and 2n comparisons on every input (slope 1), `lps_comparisons` ≤ 2m. Time: I expect KMP to win only on `worst_naive`; on `random` the naive search can be as fast or even faster because KMP does extra work (LPS array, indirection).
Plots	P1 *Recursion explosion:* calls vs n (log y), 3 input types – is it exponential? P2 *Ratio calls/cells:* recursive calls divided by n·n (log y) – how much work the memory removes. P3 *DP time:* time vs n (log-log), table vs two rows, 3 inputs – is it quadratic, does the input matter? P4 *Memory:* allocated cells vs n (log-log) – what do two rows save? P5 *Search comparisons:* comparisons vs n (log-log), naive vs KMP, 3 inputs – linear or quadratic? P6 *Search time on worst case:* time vs n (log-log) naive vs KMP, with the random input as reference – does the counter agree with the clock?

Changes to the card

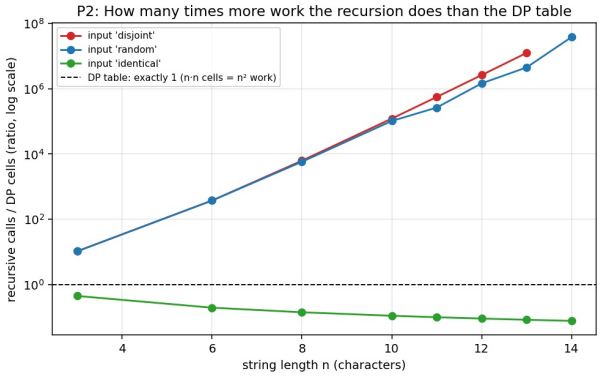
- Sizes and inputs are as planned. One rule changed: instead of "stop when a run is longer than ~8 s" (time-based, so two runs of the same command could produce different rows) the code now uses a **fixed cap** found in a trial run: recursion up to n = 13 on `disjoint` and n = 14 on `random`/`identical`, and runs with n ≥ 13 are done once instead of three times.
- Nothing else changed. Two complete runs of the final code give identical rows and identical counters (only the times differ).

2. Plots and Results

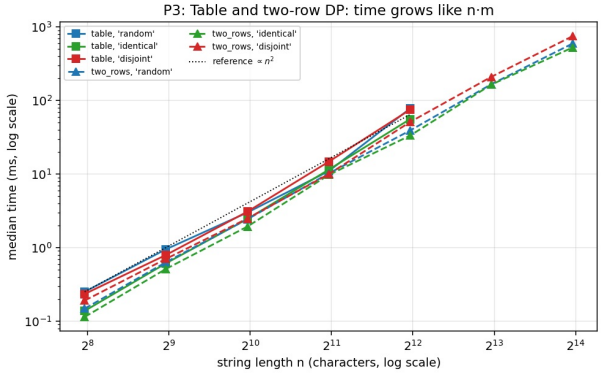
Median of 5 runs per point (3 for the recursion, 1 for n ≥ 13). Counters are exact and the same in every run.



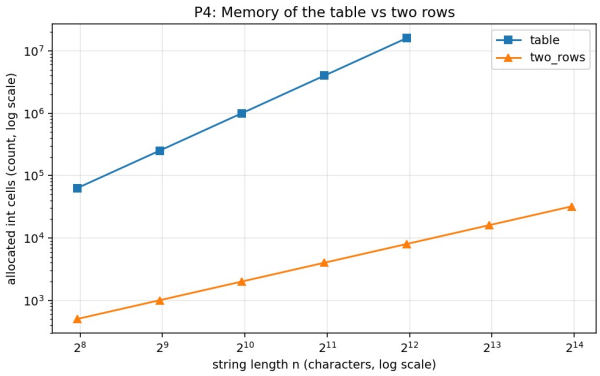
P1. On `disjoint` the calls grow ×5.6 per extra character (straight line on the log axis); on `identical` there are only n+1.



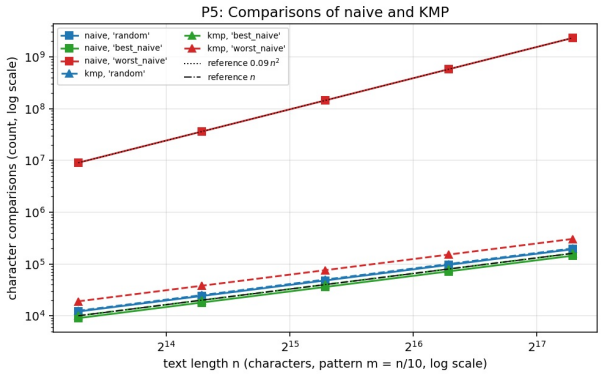
P2. At n = 13 the recursion makes 1.2·10⁷ times more calls than the table has cells (2.1·10⁹ vs 169) and the gap keeps growing.



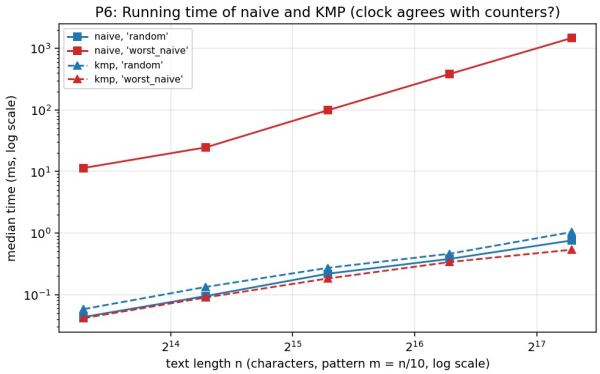
P3. Table and two rows follow the n² line, the input type matters little, and two rows are equal or faster and the only version that reaches n = 16 000.



P4. The table needs n² cells (16·10⁶ at n = 4000), two rows only 2n (8 002), so the gap grows like n/2.



P5. Naive on `worst_naive` follows 0.09·n² (2.07·10⁹ at n = 160 000); all other curves, KMP included, stay between 0.9·n and 1.9·n.



P6. On `worst_naive` naive needs 1 481 ms and KMP 0.54 ms at n = 160 000, but on `random` the two are close and naive is faster (0.76 vs 1.04 ms).

Experiment	n	Median time (counter)
Recursive, `disjoint`	12 / 13	807 ms (3.8·10⁸ calls) / 4 241 ms (2.1·10⁹ calls, 1 run)
Recursive, `random`	13 / 14	1 658 ms (7.6·10⁸ calls, 1 run) / 15 389 ms (7.5·10⁹ calls, 1 run)
Table, `random`	4 000	77.8 ms (1.6·10⁷ cells, 16 008 001 allocated ints)
Two rows, `random`	4 000 / 16 000	39.0 ms / 595 ms (2.56·10⁸ cells, 32 002 allocated ints)
Naive, `worst_naive`	160 000 (m = 16 000)	1 481 ms (2.07·10⁹ comparisons)
KMP, `worst_naive`	160 000 (m = 16 000)	0.54 ms (3.04·10⁵ comparisons = 1.9·n; LPS 3.2·10⁴)
Naive / KMP, `random`	160 000	0.76 ms (1.2·n) / 1.04 ms (1.25·n)

3. Prediction vs Result

Edit Distance. *Expected:* calls ×5.8 per character on bad input and n+1 on `identical`; `work` = n² for table and two rows on every input; time ×4 per doubling; memory n² vs 2n. *Saw:* all of it, with two differences. (1) The factor on `disjoint` is 5.56–5.60, slightly under 5.83, because the limit of the Delannoy growth is reached slowly; on `random` it jumps (3.1, 6.6, 3.6, 9.8) because every n has different strings with a different number of equal characters, so `random` is not a clean curve. (2) The time is not exactly ×4 everywhere: the table needs ×7.1 from n = 2000 to 4000 (11 → 78 ms), two rows only ×3.8. My guess, which I did not measure separately, is that the 64 MB table no longer fits in the CPU cache and must be allocated and filled for the first time. The `work` counter is exactly n² everywhere, so this is not extra work. The input type changes the DP times by up to ~40 % at the same n while the counters are identical, so I treat it as noise and branch-prediction effects.

String search. *Expected:* naive quadratic on `worst_naive`, about n on the others; KMP between n and 2n; naive possibly as fast as KMP on random text. *Saw:* naive `worst_naive` = (n−m+1)·m comparisons exactly; `random` 1.2·n (1.33 per start position × 0.9·n start positions) and `best_naive` 0.9·n. KMP: 1.0·n, 1.25·n and 1.9·n, always below 2n, and the LPS build costs at most 2m (31 997 for m = 16 000). The surprise is in the clock: KMP is fastest on `worst_naive` (0.54 ms) although it makes the most comparisons there; my guess is that the all-"A" text makes the branches predictable, while on random text they are not (not measured). The naive time on `worst_naive` grows only ×2.2 from n = 10 000 to 20 000 (JIT warm-up), then ×4.0, ×3.9 and ×3.8 as expected.

4. Short Answers

Which input made your string search slowest, and why? For naive it was `worst_naive` (text A…A, pattern A…AC): 1 481 ms at n = 160 000 against 0.76 ms on `random`. At each of the n−m+1 start positions naive compares m−1 equal characters, fails on the last one and starts again one position further, so it needs (n−m+1)·m ≈ $0.09 \cdot n^2 = 2.07 \cdot 10^9$ comparisons. KMP makes the most comparisons on the same input (1.9·n) but was not slowest in time (0.54 ms, `random` 1.04 ms), so I trust the counter for the worst case and the clock only for constant factors.

How large an n could the recursive Edit Distance handle, and what stopped it? On `disjoint` strings n = 13 (4.2 s, $2.1 \cdot 10^9$ calls), on `random` strings n = 14 once (15.4 s, $7.5 \cdot 10^9$ calls). Time stopped it, not memory, because the recursion depth is only n+m ≤ 28. Every extra character multiplies the time by about 5.6, so (extrapolated) n = 15 would need about 2 minutes and n = 20 more than a week, while the table needs 0.25 ms for n = 250. On `identical` strings the same code handles any n (n+1 calls).

What do you gain and what do you lose with two rows instead of the full table? Gain: memory falls from (n+1)(m+1) to 2(m+1) ints, 16 008 001 → 8 002 at n = 4000, and at n = 16 000 the table would need $256 \cdot 10^6$ ints (1 GB) while two rows need 32 002 and run in 0.6 s. It was also faster in my runs (39 vs 78 ms at n = 4000), probably because of the cache, and the time complexity is the same (`work` = n·m). Loss: only the number of edits is left; without the table we cannot walk back to list which insertions, deletions and replacements to do, which a "did you mean…" display with highlighted changes would need (an extra method such as Hirschberg's algorithm fixes this). The code is also a bit harder to read (row swap, string swap).

If you had to choose one string search for your city platform, which and why? KMP. On random-looking text the two are close (naive 0.76 ms, KMP 1.04 ms at 160 000 characters), but only KMP has a guarantee: never more than 2n comparisons and no backward step in the text, even for repetitive logs where naive was 6 800 times higher in comparisons (and 2 700 times slower) in my worst case. The price is an LPS array of m ints, built with at most 2m comparisons. For short street names the speed difference is tiny, so the guarantee decides.

5. One Paragraph on Time

My times range from 0.01 ms (naive, `best_naive`, n = 10 000) to 15.4 s (recursion, n = 14), but they are not my only evidence. They depend on the machine, JIT warm-up, cache and garbage collector: in an earlier complete run of the same code the recursion on `disjoint` n = 13 took 7.0 s instead of 4.2 s, and the table jump between n = 2000 and 4000 and the slow first naive runs are other examples. Below ~1 ms one reading is mostly noise, and the median of 5 runs reduces random noise but not systematic effects. The counters (calls, cells, comparisons) are exact, identical in every run and match the theory (n^2, Delannoy growth, (n−m+1)·m, ≤ 2n), so my conclusions about *growth* rest on them, and the clock is used to compare constant factors, where it sometimes disagrees with the counters (KMP on `worst_naive`).

6. Reproduce

See `README.md`. `results/results.csv` is produced by `daa.exp.Experiment` (fixed seed 2026): two runs give identical rows and counters; only `time_ms` differs.